

Exercice formatif : manipuler des données avec le tidyverse — corrigé

Chaque réponse est suivie de la notion à revoir si elle a été manquée.

Toutes les questions utilisent le tableau mesures (5 lignes) :

id	souche	temps	signal
A1	WT	0	1.2
A2	KO	0	0.8
B1	WT	24	3.5
B2	KO	24	NA
C1	WT	48	4.1

1. Le pipe

Réécrivez avec `|>` : `head(arrange(mesures, desc(signal)), 2)`. Quels id obtient-on ?

```
mesures |> arrange(desc(signal)) |> head(2)
#> # A tibble: 2 × 4
#>   id     souche temps signal
#>   <chr> <chr>   <dbl> <dbl>
#> 1 C1     WT      48     4.1
#> 2 B1     WT      24     3.5
```

On lit dans l'ordre d'exécution : « je trie, **puis** je garde les 2 premières ». À revoir : *le pipe*.

2. filter() et les NA

Combien de lignes renvoient (a) `filter(mesures, signal > 1)` et (b) `filter(mesures, signal <= 1)` ? Pourquoi la somme ne fait-elle pas 5 ?

```
filter(mesures, signal > 1) |> nrow()
#> [1] 3
filter(mesures, signal <= 1) |> nrow()
#> [1] 1
```

B2 a un signal manquant : `NA > 1` vaut `NA`, et `filter()` ne garde que les `TRUE`. B2 disparaît des deux sélections, sans message. À revoir : *filter() et les NA*.

3. Conditions multiples

On veut les mesures à 0 **ou** 24 heures. Que renvoie chaque commande ? Laquelle est juste ?

(a) `filter(mesures, temps == 0 | 24)` (b) `filter(mesures, temps %in% c(0, 24))`

```
filter(mesures, temps == 0 | 24) |> pull(id)
#> [1] "A1" "A2" "B1" "B2" "C1"
filter(mesures, temps %in% c(0, 24)) |> pull(id)
#> [1] "A1" "A2" "B1" "B2"
```

La commande (a) se lit (temps == 0) | 24 : 24 vaut TRUE, la condition est toujours vraie et C1 (48 h) est gardé. Seule la commande (b) est juste. À revoir : `filter()`, `%in%`.

4. mutate() ou summarise() ?

Combien de lignes et de colonnes renvoie chaque commande ?

- (a) `mesures |> group_by(souche) |> summarise(moy = mean(temps))`
 (b) `mesures |> group_by(souche) |> mutate(moy = mean(temps))`

```
mesures |> group_by(souche) |> summarise(moy = mean(temps))
#> # A tibble: 2 × 2
#>   souche   moy
#>   <chr> <dbl>
#> 1 KO      12
#> 2 WT      24
mesures |> group_by(souche) |> mutate(moy = mean(temps)) |> dim()
#> [1] 5 5
```

- (a) une ligne par groupe : 2 lignes, 2 colonnes. (b) autant de lignes qu'avant : 5 lignes, 5 colonnes, la moyenne de la souche répétée. À revoir : `group_by()`, `summarise()`, `mutate()`.

5. Résumer par groupe

Que renvoie `mesures |> group_by(souche) |> summarise(n = n(), m = mean(signal))` ? Comment obtenir une moyenne pour les deux souches ?

```
mesures |> group_by(souche) |> summarise(n = n(), m = mean(signal))
#> # A tibble: 2 × 3
#>   souche     n     m
#>   <chr> <int> <dbl>
#> 1 KO         2  NA
#> 2 WT         3  2.93
mesures |> group_by(souche) |> summarise(n = n(), m = mean(signal, na.rm = TRUE))
#> # A tibble: 2 × 3
#>   souche     n     m
#>   <chr> <int> <dbl>
#> 1 KO         2  0.8
#> 2 WT         3  2.93
```

La souche KO contient le NA de B2 : sa moyenne est NA. Avec `na.rm = TRUE`, elle vaut 0.8 (une seule mesure, à noter dans l'interprétation). À revoir : `summarise()`, `NA`.

6. Changer la forme

Un tableau `qpcr` a 4 lignes (une par échantillon) et 4 colonnes : `id`, `GAPDH`, `ACTB`, `TP53`. Combien de lignes et de colonnes après `pivot_longer(qpcr, -id, names_to = "gene", values_to = "ct")` ?

```
qpcr <- tibble(id = paste0("E", 1:4), GAPDH = 18:21, ACTB = 17:20, TP53 = 25:28)
pivot_longer(qpcr, -id, names_to = "gene", values_to = "ct") |> dim()
#> [1] 12 3
```

4 échantillons × 3 gènes = 12 lignes ; 3 colonnes (`id`, `gene`, `ct`). À revoir : `pivot_longer()`.

7. Jointures

Le tableau `lots` associe un lot de réactif à chaque échantillon :

id	lot
A1	L1
A2	L1
B1	L2
B1	L2
c1	L3

(a) Combien de lignes renvoie `left_join(mesures, lots, by = "id")` ? (b) Quel lot reçoit C1 ? (c) Quelle commande liste les échantillons de mesures sans lot ?

```
lots <- tibble(id = c("A1", "A2", "B1", "B1", "c1"), lot = c("L1", "L1", "L2", "L2", "L3"))
left_join(mesures, lots, by = "id")
#> # A tibble: 6 × 5
#>   id      souche temps signal lot
#>   <chr> <chr>   <dbl> <dbl> <chr>
#> 1 A1      WT      0      1.2 L1
#> 2 A2      K0      0      0.8 L1
#> 3 B1      WT     24      3.5 L2
#> 4 B1      WT     24      3.5 L2
#> 5 B2      K0     24      NA  <NA>
#> 6 C1      WT     48      4.1 <NA>
anti_join(mesures, lots, by = "id") |> pull(id)
#> [1] "B2" "C1"
```

(a) 6 lignes au lieu de 5 : B1 est en double dans `lots`, sa mesure est dupliquée. (b) NA : "`c1`" ≠ "`C1`". (c) `anti_join()`. Aucun de ces problèmes ne produit de message. À revoir : *vérifier une jointure*.